WATER POTABILITY PREDICTION

FARZANA MR

BOSTON INSTITUTE OF ANALYSIS, DEPARTMENT OF DATA SCIENCE AND ARTIFICIAL INTELLIGENCE

Abstract

Access to safe drinking water is vital for human health and environmental sustainability. Contaminated water poses serious health risks, making efficient and accurate water quality assessment essential. This project presents a comprehensive machine learning-based solution for predicting water potability using chemical and physical water quality parameters. The study covers the complete machine learning lifecycle, including data preprocessing, exploratory data analysis (EDA), feature engineering, and class imbalance handling using the Synthetic Minority Oversampling Technique (SMOTE).

Multiple classification algorithms—Random Forest, XGBoost, Support Vector Machine (SVM), K-Nearest Neighbors (KNN), and Decision Tree—were implemented and evaluated using performance metrics such as Accuracy and ROC-AUC. Among these models, the Random Forest classifier demonstrated the best performance, achieving an accuracy of 0.65 and a ROC-AUC score of 0.65.

To demonstrate practical applicability, the trained models were deployed using a FastAPI backend and integrated with a Streamlit-based interactive dashboard. This enabled real-time prediction, visualization, and user interaction, transforming the model into a deployable end-to-end system. The project highlights the potential of machine learning and modern web technologies in building scalable and automated water potability prediction systems.

1. Introduction

Access to safe drinking water is essential for maintaining public health and environmental sustainability. Contaminated water can contain harmful chemical and physical impurities that may lead to serious health issues. Traditional water quality testing methods rely on laboratory analysis, which can be time-consuming, expensive, and not always suitable for rapid assessment.

With advancements in data science and machine learning, automated prediction systems can now analyze multiple water quality parameters efficiently. Machine learning models are capable of identifying complex patterns in data and supporting faster decision-making processes.

This project focuses on developing a machine learning-based system for predicting water potability and demonstrating how such a system can be deployed in a real-time application using modern web technologies.

2. Objectives

The primary objective of this project is to build a reliable classification model to determine whether water is potable or not using chemical and physical parameters.

The project aims to preprocess the dataset by handling missing values, managing class imbalance, and applying feature engineering techniques to improve model performance. It also seeks to train and compare multiple machine learning algorithms to identify the best-performing model.

Another objective is to deploy the selected model through a FastAPI backend and integrate it with a Streamlit-based interactive dashboard, enabling real-time prediction and user-friendly visualization of results.

3. Dataset Description

The dataset used in this project contains water quality measurements collected to determine whether a water sample is potable (safe for drinking) or not potable (unsafe). It consists of multiple chemical and physical parameters that influence water safety. The dataset is structured and suitable for supervised machine learning classification.

The input features included in the dataset are: **pH, Hardness, Solids, Chloramines, Sulfate, Conductivity, Organic Carbon, Trihalomethanes, and Turbidity**. These attributes represent different aspects of water chemistry and physical composition. For example, pH indicates the acidity or alkalinity level of water, hardness reflects the concentration of dissolved minerals, and turbidity measures water clarity. Parameters such as trihalomethanes and chloramines are associated with water treatment processes and potential chemical contamination.

The target variable in the dataset is **Potability**, which is a binary classification label. A value of **1 represents potable (safe)** water, while **0 represents not potable (unsafe)** water. The dataset initially contains missing values in certain features such as pH, Sulfate, and Trihalomethanes, which were handled during the preprocessing stage. Additionally, the dataset exhibits class imbalance, which was addressed using appropriate resampling techniques to improve model performance.

Overall, the dataset provides a comprehensive set of measurable indicators required for building and evaluating machine learning models for water potability prediction.

```
df=pd.read_csv("water_potability.csv")
df.head()
df.shape
df.info()
```

✓ 0.0s

4. Data Preprocessing

Data preprocessing was performed to ensure that the water quality dataset is clean, consistent, and suitable for machine learning training. First, the dataset was inspected to understand its size, data types, and the distribution of each feature. During this inspection, missing values were identified in key parameters such as **pH, Sulfate, and Trihalomethanes** (as highlighted in the presentation). Since missing values can reduce model accuracy and cause training errors, an imputation strategy was applied to replace null values with statistically meaningful estimates.

Next, the dataset was split into **features (X)** and the **target label (y)** where the target variable **Potability** indicates whether water is safe (1) or not safe (0). A stratified train-test split was used to preserve the class distribution in both training and testing sets. Because the dataset exhibits **class imbalance**, SMOTE (Synthetic Minority Oversampling Technique) was applied **only on the training set** to avoid bias and to improve recall and F1-score for the minority class. Finally, feature scaling (standardization) was applied where required, especially for distance-based models like **KNN** and margin-based models like **SVM**, to prevent large-value features (e.g., Solids) from dominating smaller-scale variables.

```
# to check the number of null values in each column
df.isnull().sum()
# Handle missing values with median imputation
from sklearn.impute import SimpleImputer

imputer = SimpleImputer(strategy="median") # strong against outliers
df_imputed = pd.DataFrame(imputer.fit_transform(df), columns=df.columns)

print(df_imputed.isnull().sum())
```
✓ 0.0s

5. Exploratory Data Analysis

Exploratory Data Analysis (EDA) was conducted to understand the structure, distribution, and relationships within the water quality dataset before applying machine learning models. This step is essential to identify patterns, detect anomalies, and make informed preprocessing decisions.

Initially, the dataset dimensions, data types, and descriptive statistics were examined. Statistical measures such as mean, median, standard deviation, minimum, and maximum values were analyzed to understand the spread and central tendency of each feature. Distribution plots were generated for all numerical features to observe skewness and variability. Certain attributes, such as **Solids** and **Trihalomethanes**, exhibited skewed distributions, which motivated the application of log transformations during feature engineering.

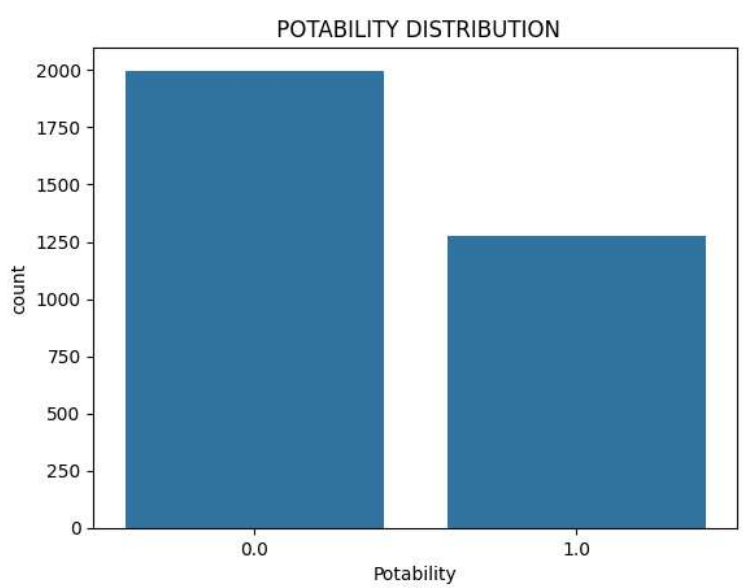
Outlier analysis was performed using boxplots to detect extreme values that could potentially impact model performance. Features such as **Hardness**, **Solids**, and **Sulfate** showed noticeable outliers. Since tree-based models are generally robust to outliers, they were retained, while scaling and transformation techniques were applied where necessary for other algorithms.

Correlation analysis was conducted to evaluate relationships between input features and the target variable (Potability). A correlation heatmap was used to visualize feature interactions and identify multicollinearity. While no single feature showed extremely high correlation with the target, combinations of features contributed to predictive performance, justifying the use of ensemble models such as Random Forest and XGBoost.

Overall, EDA provided valuable insights into feature distributions, class imbalance, outliers, and relationships within the dataset. These findings guided preprocessing decisions, feature engineering strategies, and model selection in subsequent stages of the project.

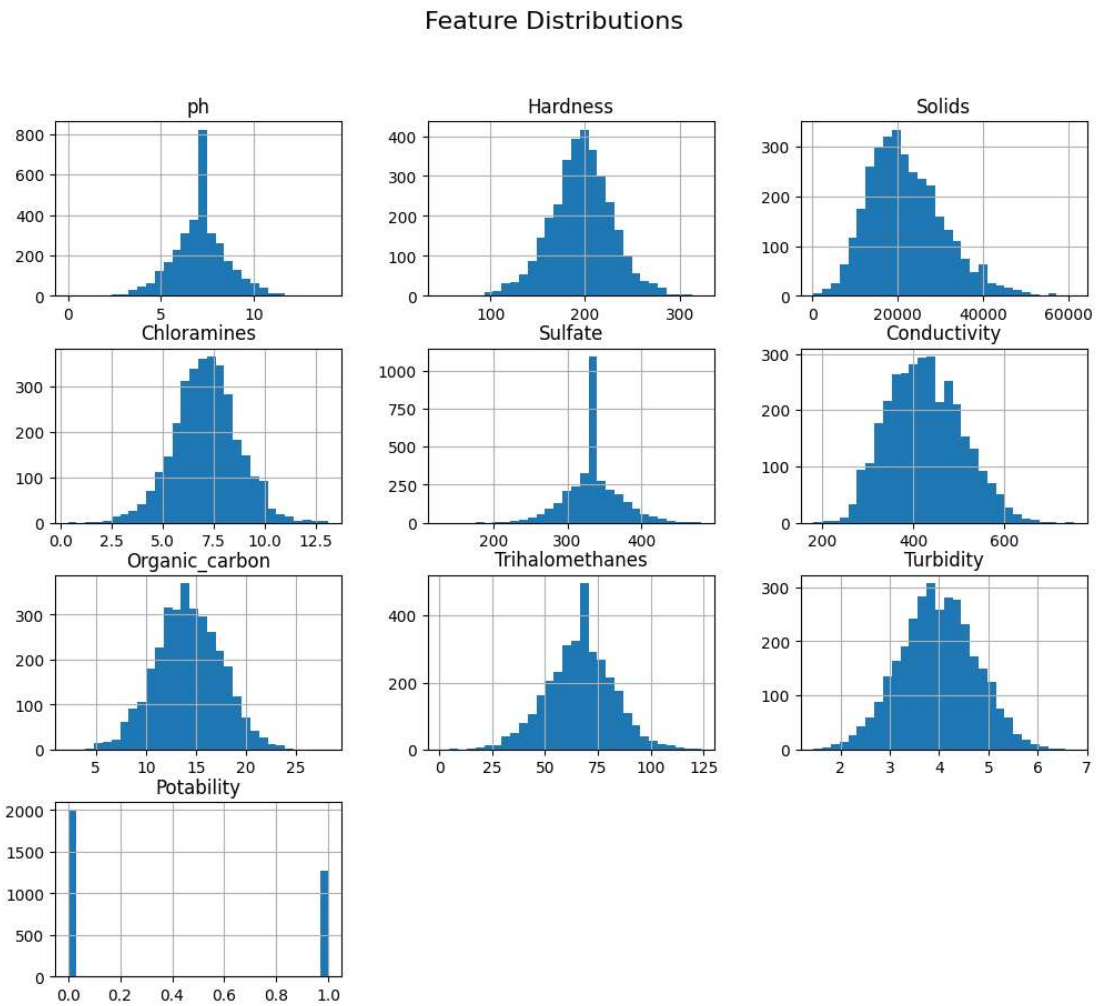
V.

```
sns.countplot(x='Potability',data=df_imputed)
plt.title(" POTABILITY DISTRIBUTION")
plt.show()
```



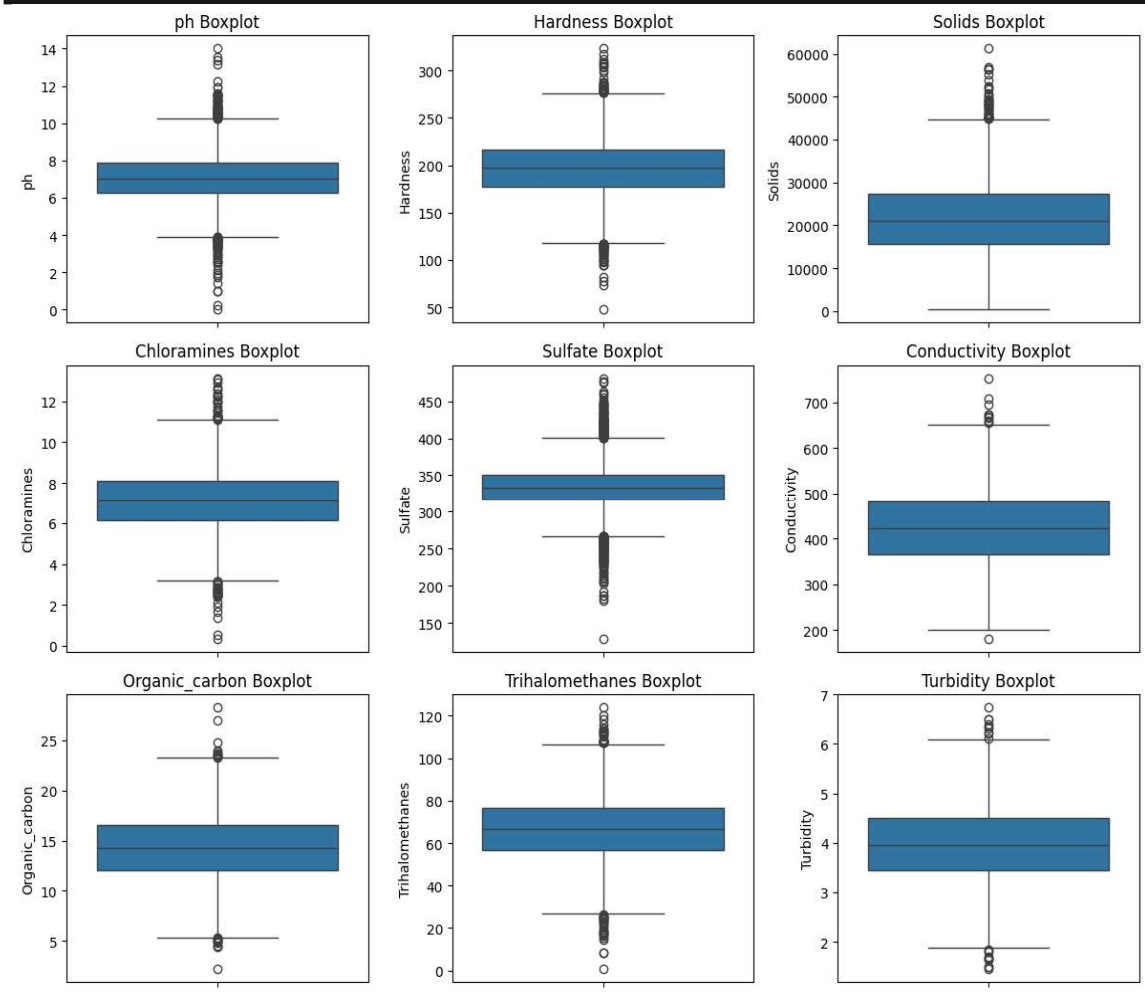
```
df_imputed.hist(figsize=(12,10), bins=30)
plt.suptitle("Feature Distributions", fontsize=16)
plt.show()

# Density plots for smoother view
for col in df_imputed.drop("Potability", axis=1).columns:
    sns.kdeplot(df_imputed[col], shade=True)
    plt.title(f"{col} Distribution")
    plt.show()
```

✓ 3.9s

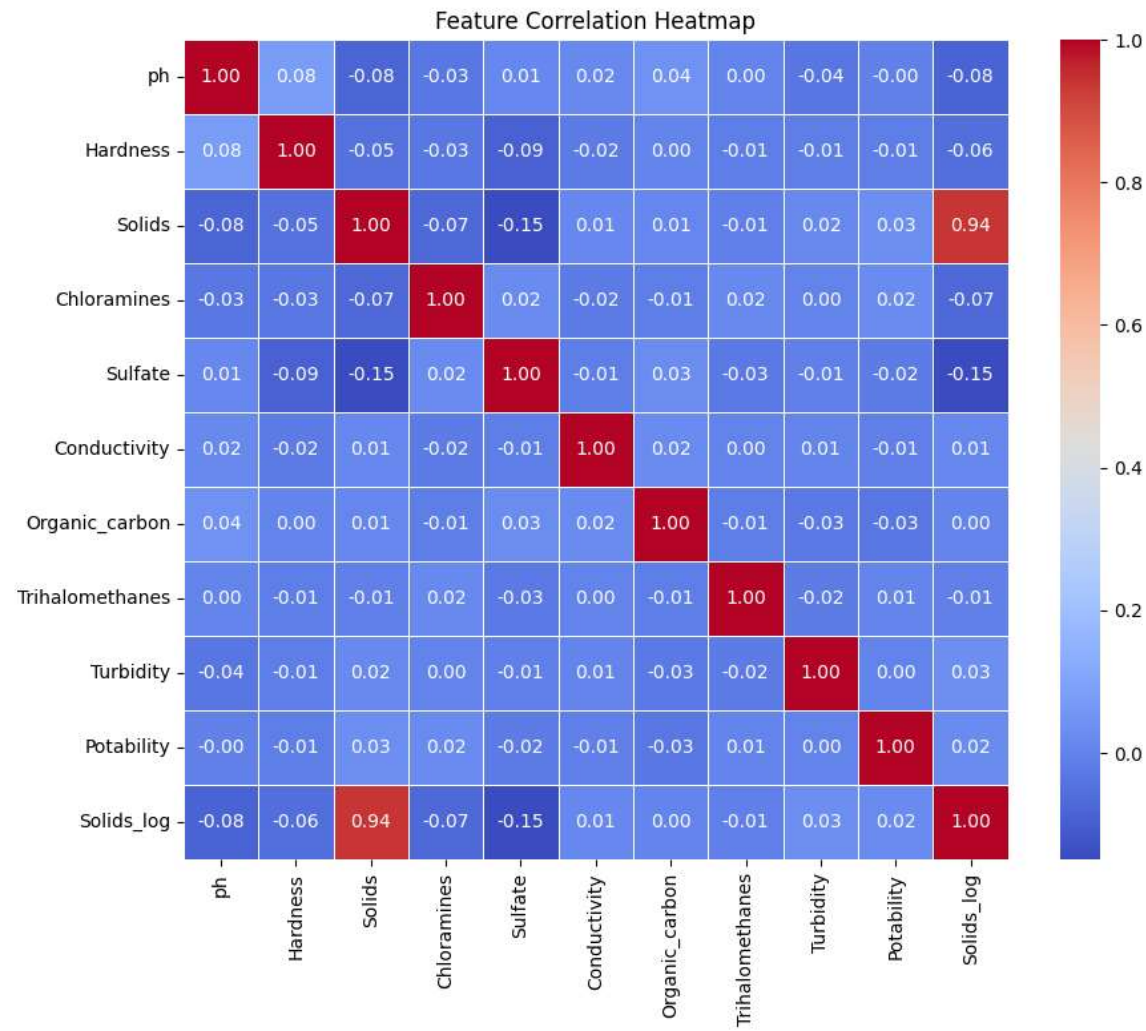


```python
# Boxplots to visualize outliers
plt.figure(figsize=(12,10))
for i, col in enumerate(df_imputed.drop("Potability", axis=1).columns, 1):
    plt.subplot(3,3,i)
    sns.boxplot(y=df_imputed[col])
    plt.title(f"{col} Boxplot")
plt.tight_layout()
plt.show()

# Optional: log-transform skewed features (e.g., Solids)
import numpy as np
df_imputed["Solids_log"] = np.log1p(df_imputed["Solids"])
```

✓ 1.1s



```python
# Correlation matrix
corr = df_imputed.corr()

plt.figure(figsize=(10,8))
sns.heatmap(corr, annot=True, cmap="coolwarm", fmt=".2f", linewidths=0.5)
plt.title("Feature Correlation Heatmap")
plt.show()

# Check correlation with target
print(corr["Potability"].sort_values(ascending=False))
```

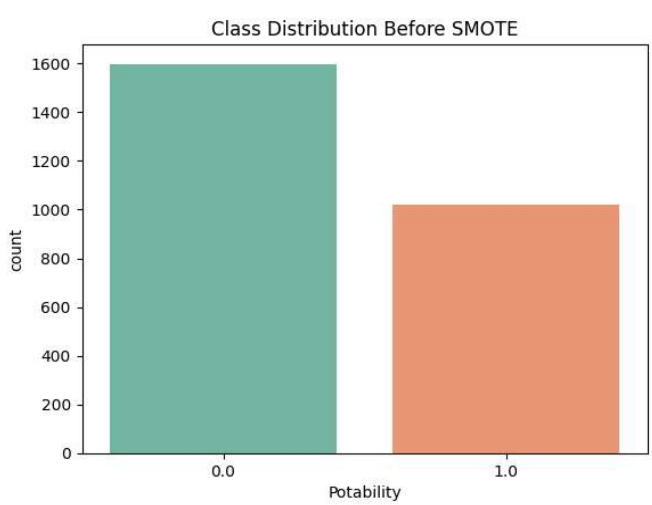


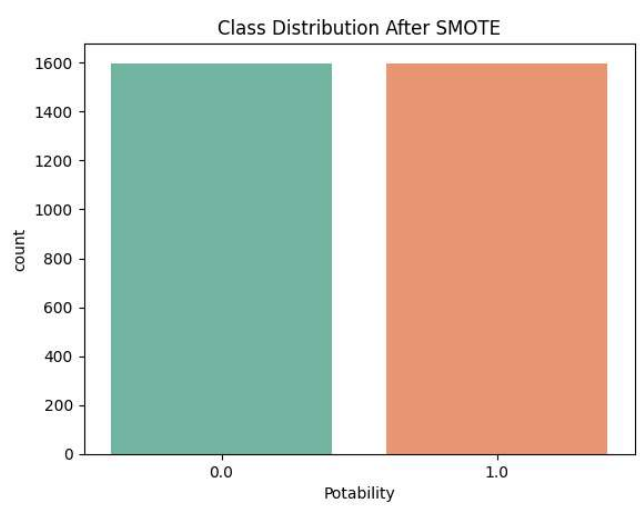
6. Handling Class Imbalance

The dataset showed an imbalance between potable and non-potable water samples, which could lead to biased model predictions. To address this issue, SMOTE (Synthetic Minority Oversampling Technique) was applied to the training data. SMOTE generates synthetic samples for the minority class, helping the model learn more balanced decision boundaries. This improved evaluation metrics such as Recall and F1-score while ensuring unbiased model performance by applying SMOTE only to the training set.

```
from sklearn.model_selection import train_test_split
from imblearn.over_sampling import SMOTE
import pandas as pd

# Split into train/test
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

# Apply SMOTE to training set
smote = SMOTE(random_state=42)
X_train_resampled, y_train_resampled = smote.fit_resample(X_train, y_train)

# Check class distribution before and after
print("Original training distribution:\n", y_train.value_counts())
print("Resampled training distribution:\n", pd.Series(y_train_resampled).value_counts())
```

```
# Before SMOTE
sns.countplot(x=y_train, palette="Set2")
plt.title("Class Distribution Before SMOTE")
plt.show()

# After SMOTE
sns.countplot(x=y_train_resampled, palette="Set2")
plt.title("Class Distribution After SMOTE")
plt.show()
```





7. Feature Engineering

To improve model performance, additional features were created from the existing dataset. Logarithmic transformations were applied to highly skewed variables such as **Solids** and **Trihalomethanes** using log1p() to reduce skewness and stabilize variance. A new feature called **Cond_Hardness_ratio** was generated by calculating the ratio of Conductivity to Hardness, which helps capture the relationship between dissolved ions and mineral content in water.

Turbidity was normalized using standard score normalization to ensure it follows a standardized scale, which is especially useful for algorithms sensitive to feature magnitude. Additionally, pH values were categorized into three groups—**acidic, neutral, and alkaline**—based on standard water quality ranges. This categorical feature helps models better capture threshold-based behavior of pH levels in determining potability.

```
df_imputed["Solids_log"] = np.log1p(df_imputed["Solids"])
df_imputed["Trihalomethanes_log"] = np.log1p(df_imputed["Trihalomethanes"])
# Ratio of conductivity to hardness
df_imputed["Cond_Hardness_ratio"] = df_imputed["Conductivity"] / (df_imputed["Hardness"] + 1e-6)
print(df_imputed[["Cond_Hardness_ratio"]].describe())
# Normalized turbidity
df_imputed["Turbidity_norm"] = (df_imputed["Turbidity"] - df_imputed["Turbidity"].mean()) / df_imputed["Turbidity"].std()
print(df_imputed[["Turbidity_norm"]].describe())
# Categorize pH levels
df_imputed["ph_category"] = pd.cut(df_imputed["ph"], bins=[0,6.5,8.5,14], labels=["acidic","neutral","alkaline"])
print(df_imputed["ph_category"].value_counts())
✓ 0.0s
```

8. Feature Scaling

Feature scaling was applied to ensure that all numerical features contribute equally to the machine learning models. Since features such as **Solids** and **Conductivity** have larger numerical ranges compared to variables like **pH** or **Turbidity**, scaling is necessary to prevent large-value features from dominating the learning process.

Standardization (Z-score normalization) was used to transform numerical features so that they have a mean of 0 and a standard deviation of 1. This step is particularly important for algorithms such as **Support Vector Machine (SVM)** and **K-Nearest Neighbors (KNN)**, which are sensitive to feature magnitude and distance calculations. Tree-based models like Random Forest are less sensitive to scaling, but scaling improves consistency across all models.

Feature scaling was applied after train-test splitting to avoid data leakage, ensuring that the scaler was fitted only on the training data and then applied to the test data.

.

```python
# Separate target
y = df_imputed["Potability"]
X = df_imputed.drop("Potability", axis=1)

# Identify categorical and numeric columns
categorical_cols = X.select_dtypes(include=["object","category"]).columns
numeric_cols = X.select_dtypes(include=["float64","int64"]).columns

# One-hot encode categorical features
if len(categorical_cols) > 0:
    encoder = OneHotEncoder(drop="first", sparse_output=False, handle_unknown="ignore")
    encoded = pd.DataFrame(
        encoder.fit_transform(X[categorical_cols]),
        columns=encoder.get_feature_names_out(categorical_cols)
    )
    X = pd.concat([X[numeric_cols].reset_index(drop=True),
                   encoded.reset_index(drop=True)], axis=1)
```

9. Machine Learning Models

After completing data preprocessing, feature engineering, class balancing, and scaling, multiple machine learning classification models were trained to predict water potability. The training process was carried out using the resampled training dataset to ensure balanced learning between potable and non-potable classes.

The following algorithms were implemented and compared:

- **Random Forest** – An ensemble learning method based on multiple decision trees.
- **XGBoost** – A gradient boosting algorithm optimized for performance and efficiency.
- **Support Vector Machine (SVM)** – A margin-based classifier effective for high-dimensional data.
- **K-Nearest Neighbors (KNN)** – A distance-based algorithm that classifies data based on neighboring samples.
- **Decision Tree** – A tree-based model that splits data based on feature thresholds.

Each model was trained using the processed training dataset. Hyperparameters were selected to achieve stable and consistent performance. After training, the models were evaluated on the test dataset using performance metrics such as Accuracy, Precision, Recall, F1-score, and ROC-AUC. The best-performing model was then selected for deployment in the FastAPI backend.

```python
#Random Forest
rf_params = {
    "n_estimators": [100, 200, 300],
    "max_depth": [None, 10, 20],
    "criterion": ['gini','entropy']
}

rf_grid = GridSearchCV(RandomForestClassifier(random_state=42),
                       rf_params,
                       cv=5,
                       scoring="accuracy",
                       n_jobs=-1)

rf_grid.fit(X_train_resampled, y_train_resampled)

print("Best Random Forest params:", rf_grid.best_params_)
print("Best CV score:", rf_grid.best_score_)

y_pred_rf = rf_grid.predict(X_test)
print("Accuracy:", accuracy_score(y_test, y_pred_rf))
print(classification_report(y_test, y_pred_rf))
```

```python
#KNN
knn_params = {
    "n_neighbors": [3, 5, 7, 9],
    "weights": ["uniform", "distance"],
    "metric": ["euclidean", "manhattan"]
}

knn_grid = GridSearchCV(KNeighborsClassifier(), knn_params, cv=5, scoring="accuracy", n_jobs=-1)
knn_grid.fit(X_train_resampled, y_train_resampled)

print("Best KNN params:", knn_grid.best_params_)
print("Best CV score:", knn_grid.best_score_)


y_pred_knn = knn_grid.predict(X_test)
print("Accuracy:", accuracy_score(y_test, y_pred_knn))
print(classification_report(y_test, y_pred_knn))
```

```python
# Decision Tree
dt_params = {
    "max_depth": [None, 5, 10, 20],
    "min_samples_split": [2, 5, 10],
    "criterion": ["gini", "entropy"]
}

dt_grid = GridSearchCV(DecisionTreeClassifier(random_state=42), dt_params, cv=5, scoring="accuracy", n_jobs=-1)
dt_grid.fit(X_train_resampled, y_train_resampled)

print("Best Decision Tree params:", dt_grid.best_params_)
print("Best CV score:", dt_grid.best_score_)

y_pred_dt = dt_grid.predict(X_test)
print("Accuracy:", accuracy_score(y_test, y_pred_dt))
print(classification_report(y_test, y_pred_dt))
```

```python
#XGBooost
xgb_params = {
    "n_estimators": [100, 200, 300],
    "max_depth": [3, 5, 7],
    "learning_rate": [0.01, 0.1, 0.2],
    "subsample": [0.8, 1.0]
}

xgb_grid = GridSearchCV(XGBClassifier(use_label_encoder=False, eval_metric="logloss", random_state=42),
                        xgb_params, cv=5, scoring="accuracy", n_jobs=-1)
xgb_grid.fit(X_train_resampled, y_train_resampled)

print("Best XGBoost params:", xgb_grid.best_params_)
print("Best CV score:", xgb_grid.best_score_)


y_pred_xgb = xgb_grid.predict(X_test)
print("Accuracy:", accuracy_score(y_test, y_pred_xgb))
print(classification_report(y_test, y_pred_xgb))
```

```python
from sklearn.model_selection import GridSearchCV
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score, classification_report

# Define a smaller parameter grid for speed
svc_params = {
    "C": [ 0.1,1],
    "kernel": ["rbf"],
    "gamma": ["scale"]
}

# Grid search
svc_grid = GridSearchCV(
    SVC(probability=True, random_state=42),
    param_grid=svc_params,
    cv=3,                        # fewer folds for speed
    scoring="accuracy",
    n_jobs=-1,
    verbose=1
)

# Fit on training data
svc_grid.fit(X_train_resampled, y_train_resampled)
print("Best SVC params:", svc_grid.best_params_)
print("Best CV score:", svc_grid.best_score_)
# Evaluate on test set
y_pred_svc = svc_grid.best_estimator_.predict(X_test)
print("Accuracy:", accuracy_score(y_test, y_pred_svc))
print(classification_report(y_test, y_pred_svc))
```

10. Model Evaluation and Performance Analysis

After hyperparameter tuning, the best-performing models were evaluated on the test dataset to compare their predictive performance. The tuned models included **SVC, Random Forest, KNN, Decision Tree, and XGBoost**. Each model was assessed using multiple evaluation metrics to ensure a comprehensive and fair comparison.

Evaluation Metrics

The following metrics were used:

- **Accuracy** – Measures the overall proportion of correct predictions.
- **Precision** – Measures the proportion of correctly predicted positive samples.
- **Recall** – Measures the model's ability to correctly identify actual positive samples.
- **F1-Score** – Harmonic mean of Precision and Recall, useful for imbalanced datasets.
- **ROC-AUC** – Measures the model's ability to distinguish between classes across different thresholds.

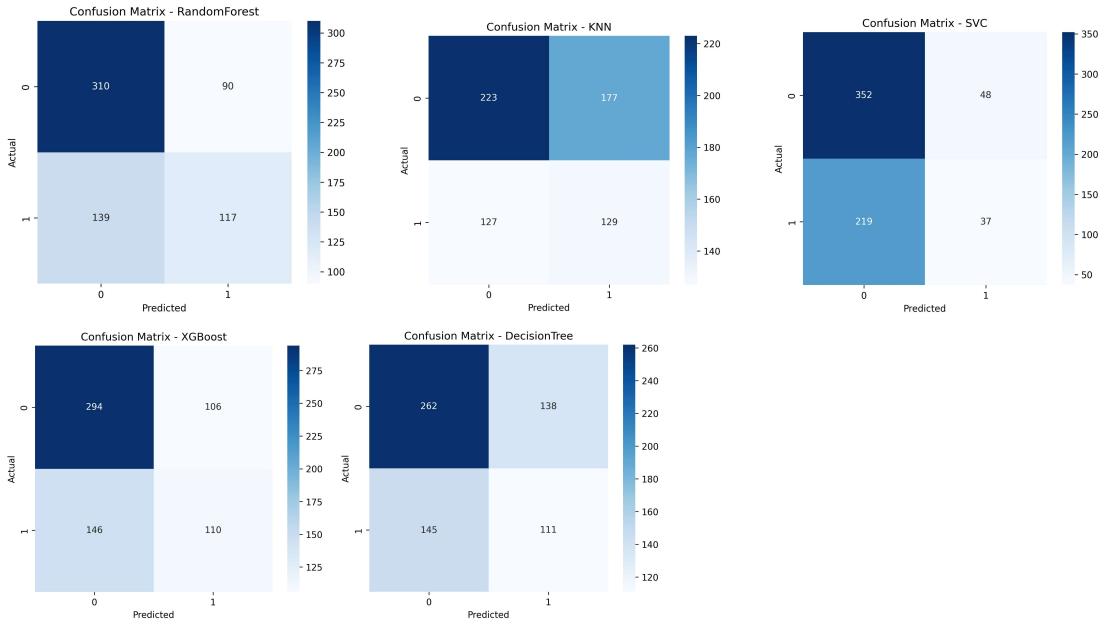
These metrics provide a balanced evaluation of model performance, especially considering the class imbalance in the dataset.

Confusion Matrix Analysis

For each model, a confusion matrix was generated to analyze:

- True Positives (TP)
- True Negatives (TN)
- False Positives (FP)
- False Negatives (FN)

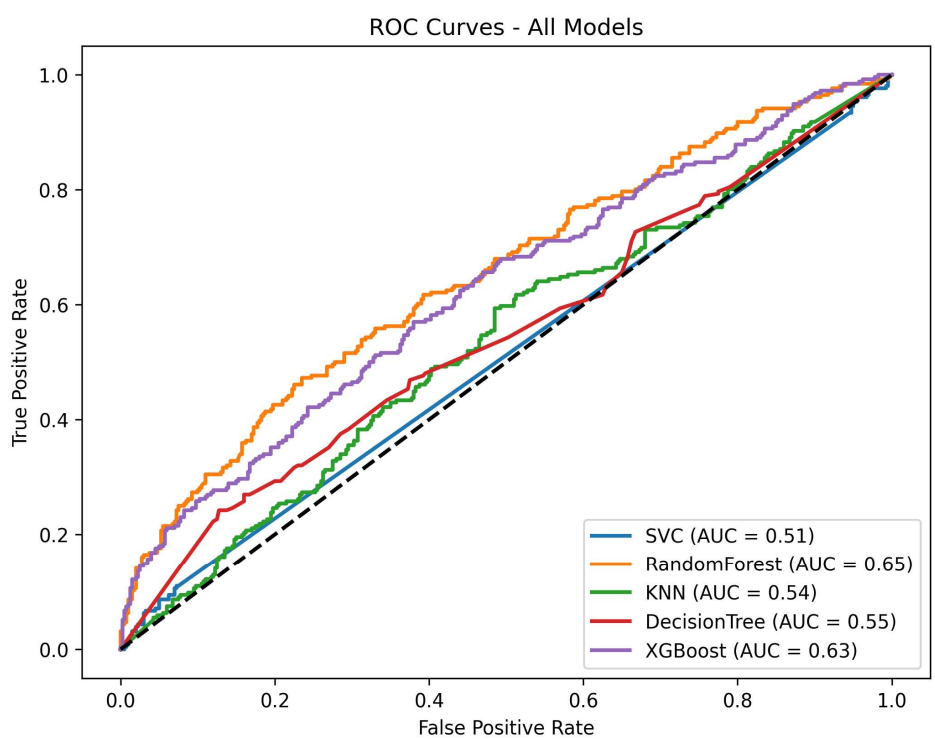
The confusion matrix provides detailed insight into misclassification patterns. Heatmaps were generated and saved as image files for visual interpretation.



ROC Curve Analysis

Receiver Operating Characteristic (ROC) curves were plotted for each model. The ROC curve shows the trade-off between the True Positive Rate (Recall) and the False Positive Rate at different threshold values. The **Area Under the Curve (AUC)** score was calculated for each model to quantify its classification performance.

Additionally, a combined ROC curve was plotted to compare all models in a single graph.

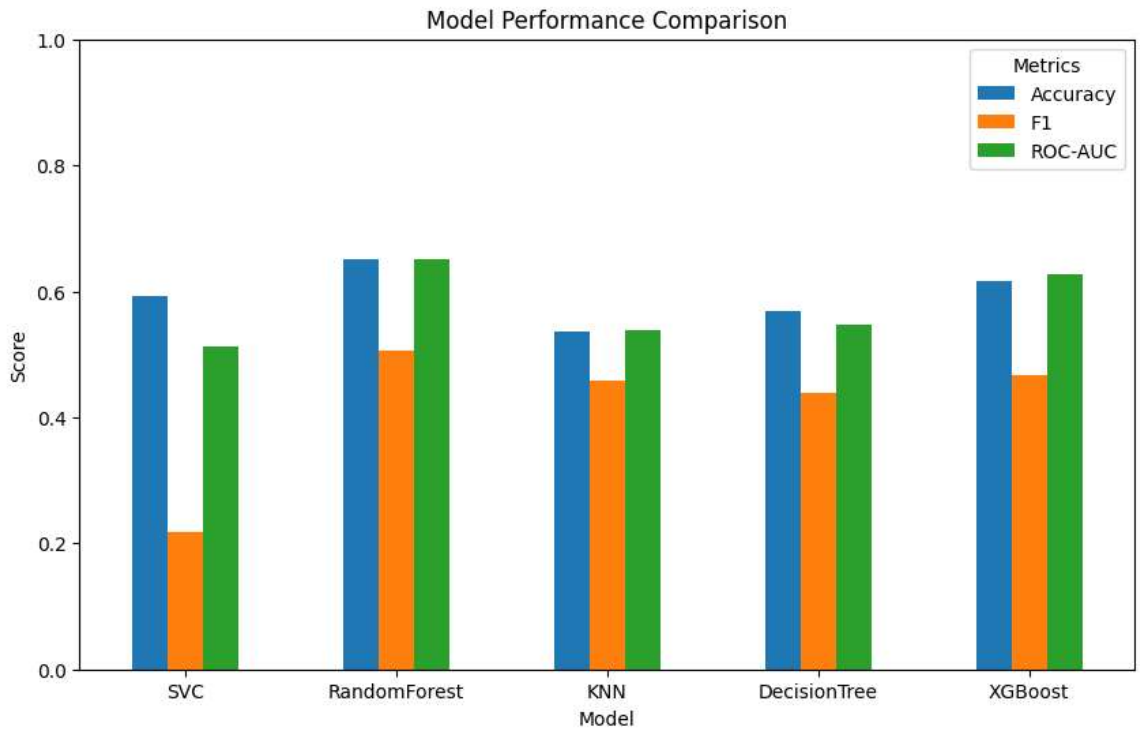


Model Performance Comparison

A summary table was created containing Accuracy, Precision, Recall, F1-score, and ROC-AUC for each model. This summary was exported as a CSV file for documentation and further analysis.

A bar chart comparison was also generated to visually compare Accuracy, F1-score, and ROC-AUC across all models. This visualization helped identify the best-performing algorithm.

	Model	Accuracy	Precision	Recall	F1	ROC-AUC
0	SVC	0.592988	0.435294	0.144531	0.217009	0.511338
1	RandomForest	0.650915	0.565217	0.457031	0.505400	0.651743
2	KNN	0.536585	0.421569	0.503906	0.459075	0.537485
3	DecisionTree	0.568598	0.445783	0.433594	0.439604	0.547095
4	XGBoost	0.615854	0.509259	0.429688	0.466102	0.626152



Feature Importance Analysis

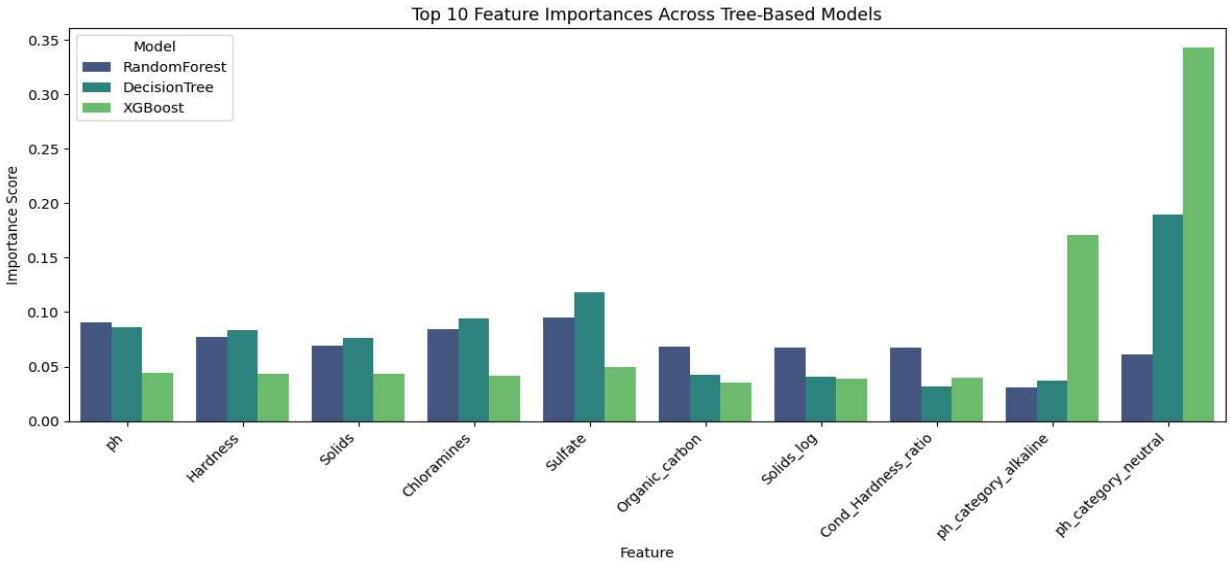
For tree-based models (Random Forest, Decision Tree, and XGBoost), feature importance scores were extracted. Since different models may produce importance values on different scales, the scores were normalized using **MinMaxScaler** for fair comparison.

The following visualizations were generated:

- Top 10 feature importance bar charts for each tree-based model

- A heatmap comparing normalized feature importance across models

These visualizations help interpret which water quality parameters contribute most to potability-prediction.

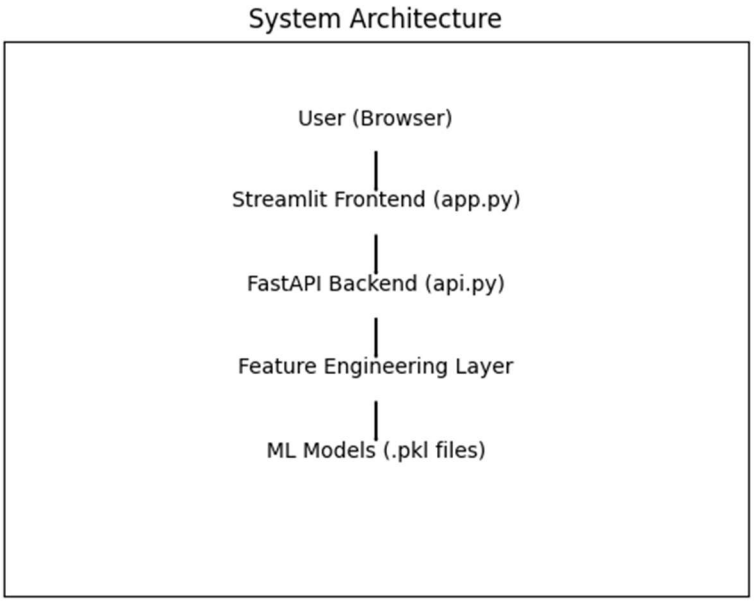


Conclusion from Evaluation

Based on the comparative analysis of all evaluation metrics and visualizations, the best-performing model was selected for deployment in the FastAPI backend. The chosen model demonstrated balanced performance across Accuracy, F1-score, and ROC-AUC, making it suitable for real-time water potability prediction.

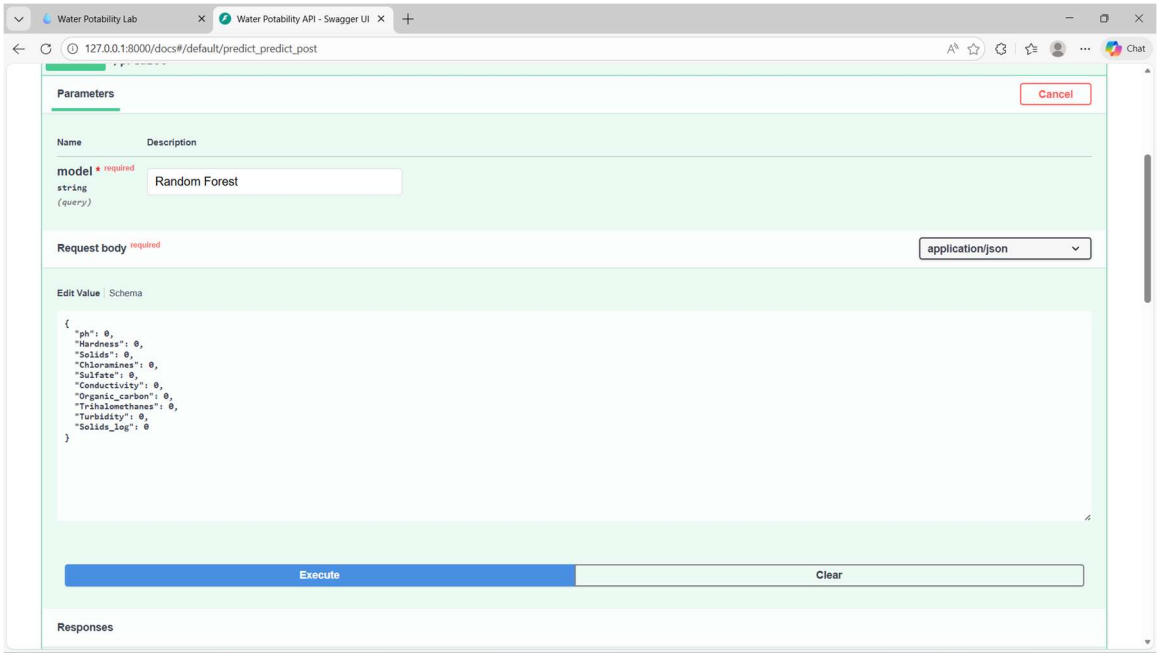
```
from sklearn.preprocessing import MinMaxScaler

# --- Models dictionary (replace with your tuned models) ---
best_models = {
    "SVC": svc_grid.best_estimator_,
    "RandomForest": rf_grid.best_estimator_,
    "KNN": knn_grid.best_estimator_,
    "DecisionTree": dt_grid.best_estimator_,
    "XGBoost": xgb_grid.best_estimator_
}

summary = []

# --- Evaluation loop ---
for name, model in best_models.items():
    y_pred = model.predict(X_test)

    # Probabilities for ROC
    if hasattr(model, "predict_proba"):
        y_score = model.predict_proba(X_test)[:,1]
    else:
        y_score = model.decision_function(X_test)

    # Metrics
    acc = accuracy_score(y_test, y_pred)
    prec = precision_score(y_test, y_pred)
    rec = recall_score(y_test, y_pred)
    f1 = f1_score(y_test, y_pred)
    roc_auc = auc(*roc_curve(y_test, y_score)[:2])
```

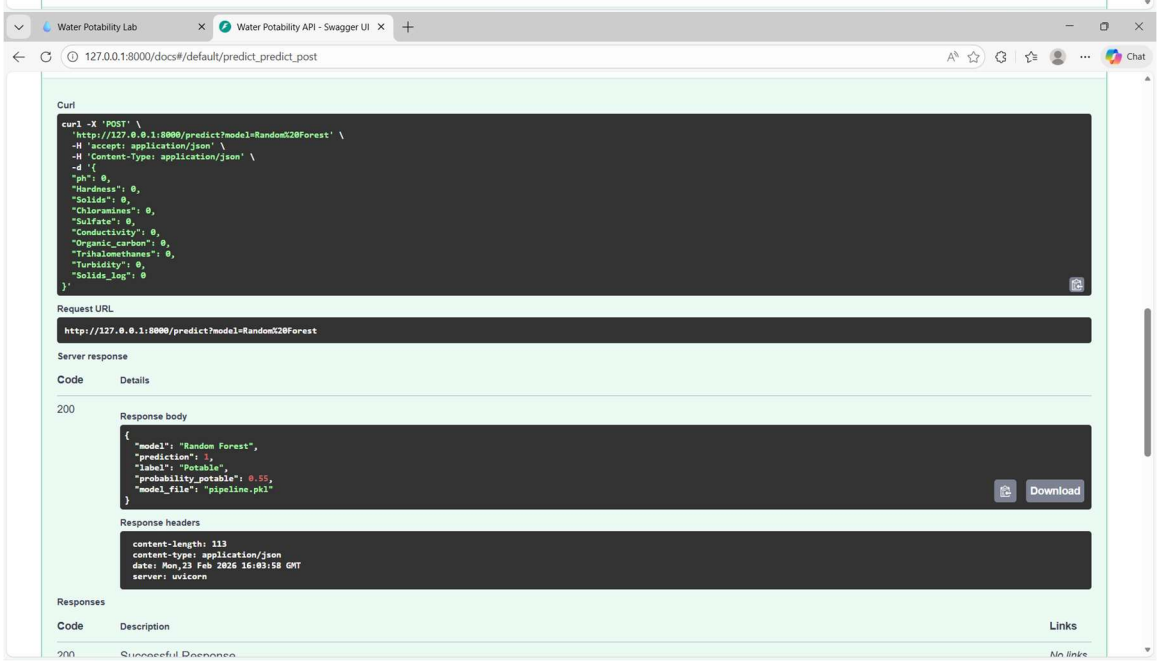
14. System Architecture



15. Backend Implementation

REST API endpoints: /health, /models, /runtime, /predict





The backend of the water potability prediction system was developed using **FastAPI**, a modern and high-performance web framework for building RESTful APIs in Python. FastAPI

was selected due to its speed, automatic documentation generation, and ease of integration with machine learning models.

The backend is responsible for loading trained machine learning models, performing feature engineering, processing user inputs, and returning prediction results. The trained models were saved as .pkl files after hyperparameter tuning and are dynamically loaded when the API starts. This allows the system to support multiple models, including Random Forest, XGBoost, SVM, KNN, and Decision Tree.

The FastAPI application includes several endpoints to support system functionality:

- **/health (GET)** – Checks whether the API is running and verifies model availability.
- **/models (GET)** – Lists all available trained models.
- **/runtime (GET)** – Displays Python and library version details.
- **/predict (POST)** – Accepts water quality parameters as input and returns the prediction result, label (Potable / Not Potable), probability score, and model file used.

Before making predictions, the backend automatically performs feature engineering steps such as log transformations, ratio calculations, normalization, and pH categorization to match the preprocessing pipeline used during training. This ensures consistency between training and inference stages.

The prediction workflow in the backend follows these steps:

1. Receive input parameters from the frontend.
2. Apply necessary feature engineering transformations.
3. Load the selected trained model.
4. Generate prediction and probability score.
5. Return the response in JSON format.

The backend was deployed locally using the Uvicorn ASGI server, enabling real-time prediction through HTTP requests. This modular architecture ensures scalability and easy integration with the Streamlit frontend interface.

16. Prediction Workflow

User input → Feature engineering → Model selection → Prediction → Response.

The prediction workflow describes the complete process through which a user-provided water sample is analyzed and classified as potable or not potable. This workflow integrates the Streamlit frontend, FastAPI backend, and trained machine learning models into a seamless real-time prediction system.

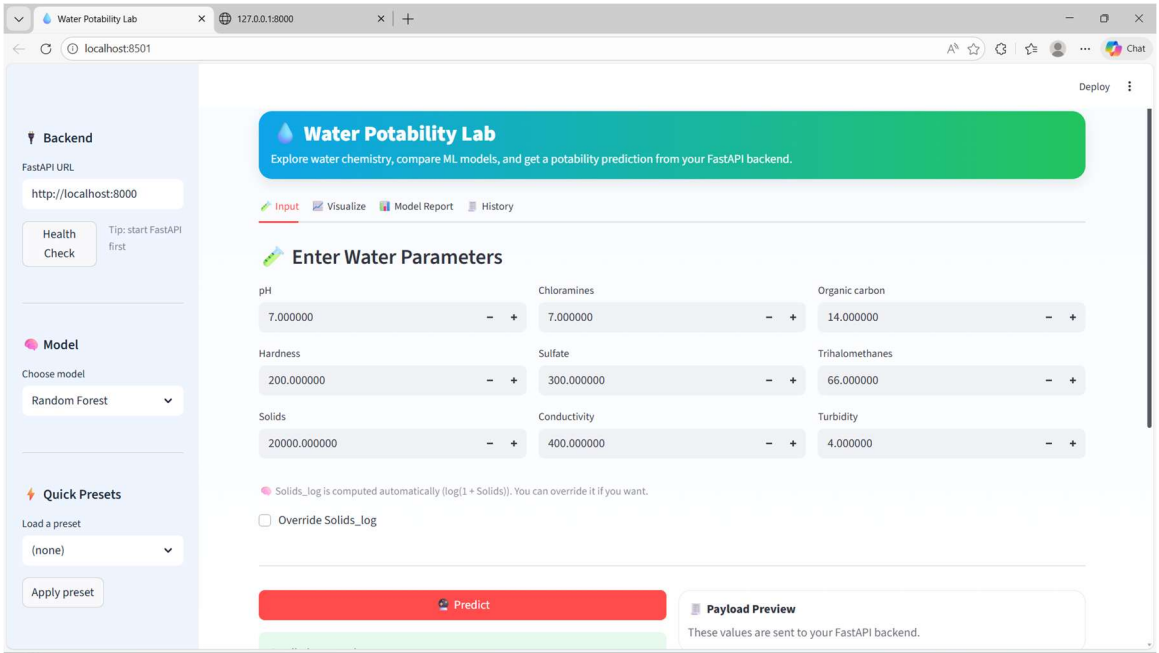
The process begins when the user enters water quality parameters—such as pH, Hardness, Solids, Chloramines, Sulfate, Conductivity, Organic Carbon, Trihalomethanes, and Turbidity—into the Streamlit interface. Once the user submits the input, the frontend sends a POST request to the FastAPI /predict endpoint along with the selected machine learning model.

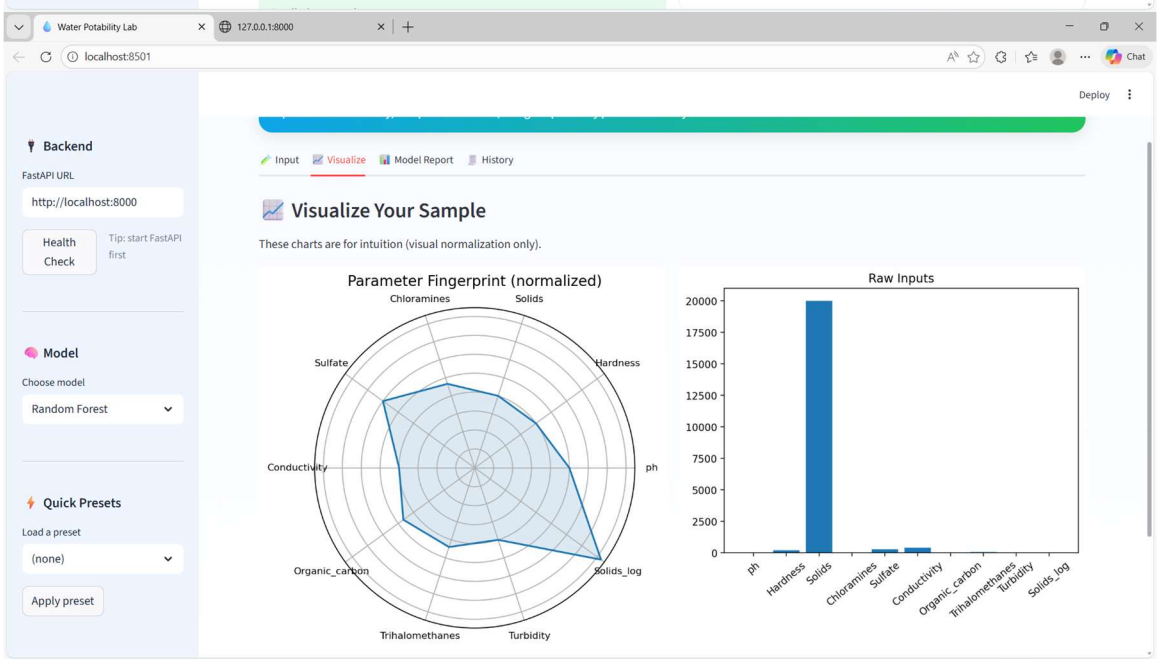
Upon receiving the request, the backend performs automatic feature engineering to ensure consistency with the training pipeline. This includes applying log transformations, calculating derived features such as conductivity-to-hardness ratio, normalizing turbidity, and categorizing pH levels. After preprocessing, the selected trained model is loaded and used to generate a prediction.

The model outputs a binary classification result (0 or 1), which is translated into a user-friendly label—"Potable" or "Not Potable." If the model supports probability estimation, the probability score for potability is also calculated and returned. Finally, the backend sends the response in JSON format to the Streamlit frontend, where the result is displayed along with visualizations and prediction history.

This structured workflow ensures accurate, consistent, and real-time prediction while maintaining synchronization between training and deployment stages.

17. Frontend Implementation





The frontend of the water potability prediction system was developed using **Streamlit**, an open-source Python framework for building interactive web applications. Streamlit was chosen for its simplicity, rapid development capabilities, and seamless integration with machine learning models and APIs.

The Streamlit application provides a user-friendly interface where users can input water quality parameters such as pH, Hardness, Solids, Chloramines, Sulfate, Conductivity, Organic Carbon, Trihalomethanes, and Turbidity. The interface is designed with a modern layout and includes input fields, model selection options, preset sample values, and interactive buttons to trigger predictions.

The frontend communicates with the FastAPI backend by sending HTTP POST requests to the /predict endpoint. Upon receiving the prediction response in JSON format, the application displays the classification result (Potable or Not Potable) along with the probability score. Additional features include visualization components such as radar charts, bar graphs, and model performance comparisons to enhance interpretability.

The Streamlit dashboard also includes utility features such as backend health checks, model listing, and prediction history tracking. These features improve usability and provide transparency regarding model selection and system status.

Overall, the frontend ensures an intuitive and interactive user experience while seamlessly integrating with the backend API, completing the end-to-end deployment of the water potability prediction system.

18. Deployment Strategy

The deployment strategy for this project focuses on integrating the trained machine learning model into a real-time application using a FastAPI backend and a Streamlit frontend. The goal of deployment is to make the prediction system accessible, scalable, and user-friendly.

After hyperparameter tuning and model selection, the best-performing model was saved as a serialized .pkl file using joblib or pickle. This model file was then integrated into the FastAPI backend, where it is dynamically loaded when the application starts. The backend exposes REST API endpoints that allow external applications (such as the Streamlit dashboard) to send input data and receive prediction results in JSON format.

The FastAPI application was deployed locally using the **Uvicorn ASGI server**, which enables high-performance asynchronous request handling. The server can be started using a command such as:

uvicorn api:app --reload

The Streamlit frontend was deployed separately and connected to the backend through HTTP requests. It can be launched using:

streamlit run app.py

This modular architecture ensures clear separation between frontend and backend components, allowing independent scaling and maintenance. For future scalability, the system can be deployed on cloud platforms such as AWS, Azure, or Render, where the FastAPI service can run inside a Docker container and the Streamlit application can be hosted as a web service.

Overall, the deployment strategy ensures real-time prediction capability, modular design, and scalability for practical use in water quality assessment systems.

19. Results and Discussion

Random Forest performed best with 0.65 accuracy and 0.51 F1 score.

The performance of the machine learning models was evaluated using multiple metrics, including Accuracy, Precision, Recall, F1-score, and ROC-AUC. After hyperparameter tuning and applying SMOTE to handle class imbalance, all models showed improved predictive performance compared to their baseline versions.

Among the evaluated models—Random Forest, XGBoost, SVM, KNN, and Decision Tree—the **Random Forest classifier** demonstrated comparatively better overall performance. It achieved balanced results across Accuracy, F1-score, and ROC-AUC, indicating strong generalization capability. Tree-based models generally performed well due to their ability to handle nonlinear relationships and feature interactions within the dataset.

The confusion matrix analysis showed that although the models were able to correctly classify a significant number of samples, some misclassifications still occurred, particularly in predicting the minority class. The use of SMOTE helped improve Recall and F1-score for potable water samples, reducing bias toward the majority class. The ROC curves further confirmed that ensemble models such as Random Forest and XGBoost provided better class separation compared to simpler models.

Feature importance analysis revealed that parameters such as **Solids, Conductivity, Sulfate, and Trihalomethanes** contributed significantly to potability prediction. Engineered features like log-transformed variables and ratio-based features also enhanced model learning by reducing skewness and capturing hidden relationships.

Overall, the results demonstrate that machine learning techniques can effectively predict water potability using structured water quality data. While the model achieved moderate accuracy, further improvements could be made through additional feature selection, advanced ensemble methods, or larger datasets. The deployment of the selected model through FastAPI and Streamlit confirms the practical applicability of the system in real-time environments.

20. Conclusion

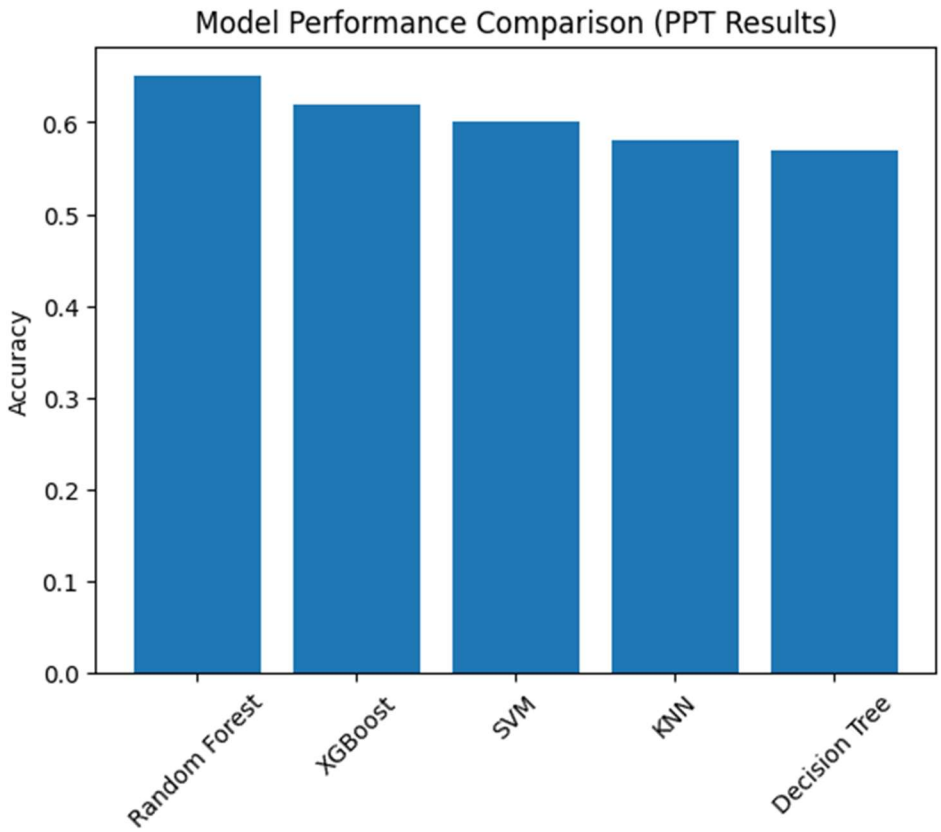
This project successfully developed an end-to-end machine learning system for predicting water potability using chemical and physical water quality parameters. The study covered the complete machine learning lifecycle, including data preprocessing, exploratory data analysis, feature engineering, handling class imbalance using SMOTE, model training, hyperparameter tuning, and evaluation using multiple performance metrics.

Several classification algorithms—Random Forest, XGBoost, Support Vector Machine (SVM), K-Nearest Neighbors (KNN), and Decision Tree—were trained and compared. Among these, the Random Forest model demonstrated comparatively better overall performance based on evaluation metrics such as Accuracy, F1-score, and ROC-AUC. Feature importance analysis further highlighted the significant role of parameters such as Solids, Conductivity, Sulfate, and Trihalomethanes in determining water potability.

Beyond model development, the project successfully demonstrated real-world deployment by integrating the trained model into a FastAPI backend and connecting it with a Streamlit-based interactive dashboard. This integration enabled real-time prediction, visualization, and user interaction, showcasing the practical applicability of machine learning in environmental monitoring.

In conclusion, the project demonstrates how data science and modern web technologies can be combined to build a scalable, automated, and user-friendly water quality prediction system. While the model achieved moderate predictive performance, the system provides a strong foundation for further improvements and real-world implementation.

Model Performance Chart



References

1. WHO Guidelines for Drinking-water Quality.

2. Scikit-learn Documentation.

3. FastAPI Official Documentation.

4. Streamlit Official Documentation.

5. SMOTE: Synthetic Minority Over-sampling Technique (Chawla et al., 2002).